

Propuesta de arquitectura 3

Cómo el sistema cumple cada necesidad del negocio, de punta a punta

Borrador para discutir en equipo · 2026-10-02 · Preparado por Harrison Tatalcha (ingesta)

En una frase: cada equipo ya construyó piezas valiosas, pero todavía no están conectadas. Esta propuesta las une con **un solo formato de mensaje, una sola puerta de entrada** (la API Java) y **un solo lugar donde se guardan los datos** (su base de datos), y muestra, necesidad por necesidad, qué está listo y qué falta construir.

1. Para qué sirve este documento

Hoy cada equipo (ingesta, backend e IA) avanzó en su parte, pero las conexiones entre las partes no están definidas: hay tres formatos de mensaje distintos y algunas piezas todavía no existen. Se analizaron tres propuestas de arquitectura:

- **Propuesta 1:** la arquitectura que se deduce de lo que ya está en las ramas del repositorio.
- **Propuesta 2:** la propuesta inicial de la ingesta (29/09), diseñada antes de conocer lo que construyó IA.
- **Propuesta 3 (la recomendada, este documento):** reutiliza todo lo construido y lo conecta.

El documento recorre las 7 necesidades del negocio (N1 a N7) y, para cada una, muestra por dónde pasa la información, dónde queda guardada y qué tan lista está hoy. **Es una propuesta para discutir:** la sección 9 reúne las preguntas que necesitamos responder entre todos.

Cómo leer los diagramas

Cada caja es una pieza del sistema; el color indica a qué equipo pertenece. La etiqueta de la esquina dice qué tan lista está la pieza para cumplir ese paso.

Discord

Personas

Programador de tareas

Ingesta (bot y lotes)

Backend: API Java

Backend: base de datos

IA

Panel y dashboard

OCI Object Storage

LISTO

 ya existe y funciona

ADAPTAR

 existe, pero hay que ajustarlo

FALTA

 hay que construirlo

2. Las necesidades del negocio: N1 a N7

Salen del brief del cliente y de las decisiones del equipo para el MVP. Son la vara con la que se mide cualquier arquitectura: **si una necesidad no se cumple, el producto no resuelve el problema del cliente.**

Código	Necesidad	Qué significa, en simple	Ejemplo	Para quién
N1	Capturar la actividad sin esfuerzo manual	Todo lo que se escribe en Discord entra al sistema solo: lo que ya pasó (historial) y lo que pasa ahora (en vivo)	Nadie copia mensajes a mano	Todo el sistema
N2	Identificar al instante los logros y las dudas valiosas	Apenas llega un mensaje, el sistema sabe si es un logro, una duda o un comentario	"me contrataron" se reconoce como logro en segundos	Marketing y CM
N3	Posts de LinkedIn con la voz de la marca	A partir de un logro, la IA escribe un borrador de post y de caso de éxito, con el tono de la institución	El logro de Camila se convierte en un post para revisar	Marketing
N4	FAQ y bot que responde dudas	El bot responde dudas en vivo usando solo los documentos de la institución. Las dudas repetidas se convierten en FAQ y tutoriales	"¿cuándo empiezan las inscripciones?" se responde citando el reglamento	Alumnos y equipo educativo

Código	Necesidad	Qué significa, en simple	Ejemplo	Para quién
N5	Dashboard de sentimiento, tendencias y alertas	Un tablero con el clima de la comunidad, los temas del momento y quién necesita apoyo	"Luis no escribe hace 14 días"	CM
N6	Aprobación humana antes de publicar	Nada se publica sin que una persona lo revise, lo edite si hace falta y lo apruebe	Marketing corrige una frase del post y lo aprueba	Marketing y CM
N7	Guardar los activos en OCI	Los activos aprobados se guardan en un bucket de Oracle Cloud. Es un requisito técnico del brief	El post aprobado queda como archivo en OCI	Requisito del brief

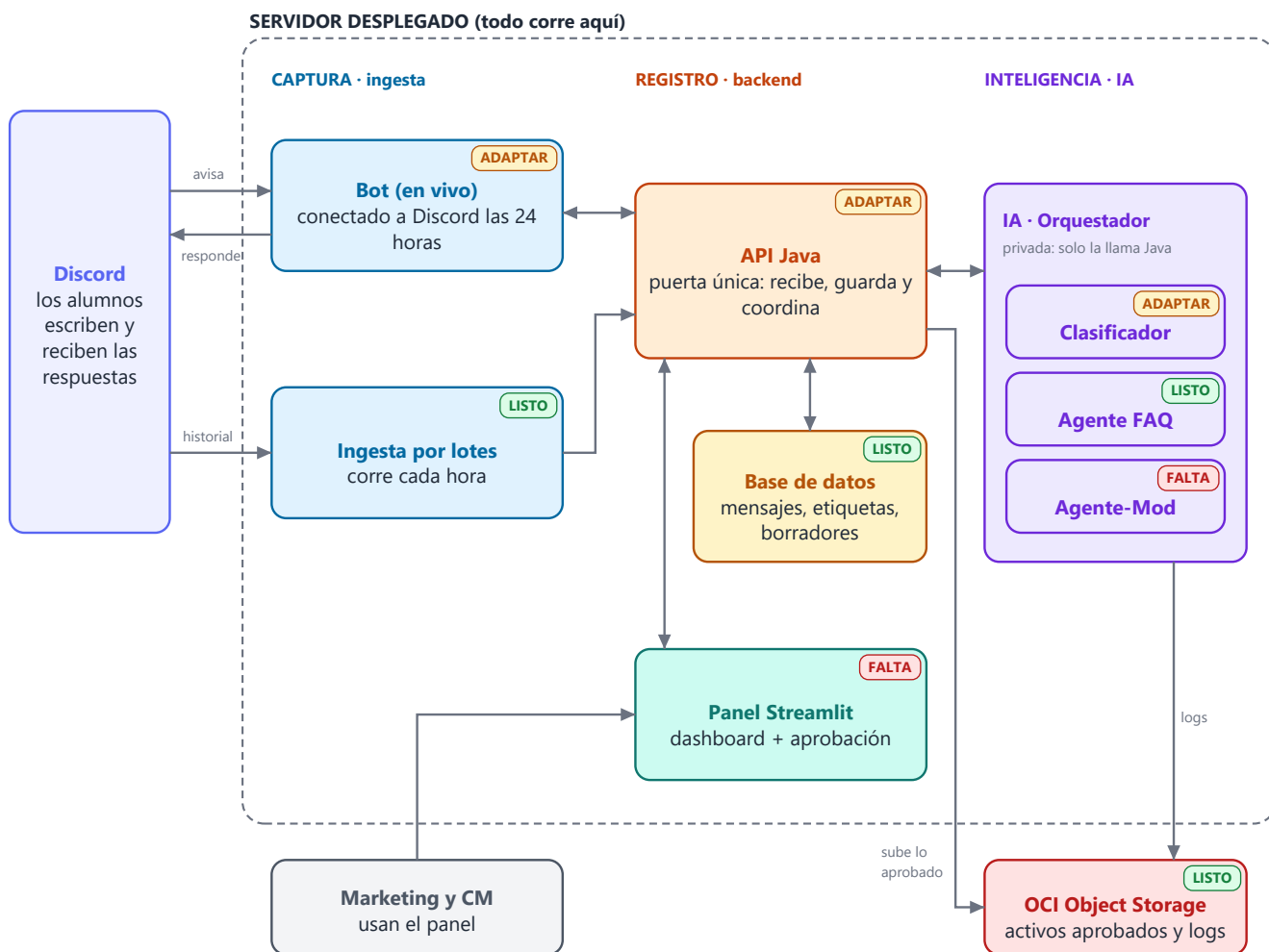
Fuera del MVP (decisión del equipo, 2026-10-01): publicaciones para X y newsletters (solo LinkedIn), y el resumen semanal Community Highlights. Solo se trabaja con Discord.

3. Términos que aparecen en el documento

Término	Qué significa	En el proyecto
API	Un programa que recibe pedidos de otros programas por "puertas" (endpoints) y les responde	La API Java de backend
Base de datos	Un archivador con fichas ordenadas: se buscan y combinan datos al instante	PostgreSQL, que maneja la API Java
OCI Object Storage	Una bodega de archivos en la nube de Oracle (un "bucket"): se guarda y se entrega cada archivo completo	Donde quedan los activos aprobados
Bot	Un programa con cuenta propia en Discord que escucha mensajes y responde	El bot que hoy tiene IA
Contrato de mensaje	El formato acordado en que viaja cada mensaje de Discord por el sistema	El contrato v1 de la ingesta (23 campos)
Lote	Un envío con uno o muchos mensajes	La ingesta envía un lote cada hora
Orquestador	El programa de IA que decide qué es cada mensaje y se lo pasa al agente que corresponde	En la carpeta agents/orquestador
Agente	Un programa con IA especializado en una tarea	Agente FAQ (dudas), Agente-Mod (posts y casos de éxito)
RAG	Que la IA responda solo con información de documentos dados, sin inventar	El Agente FAQ usa 12 PDFs de la institución
Etiquetas	Lo que la IA concluye de un mensaje	Intención, sentimiento y tema
Upsert	"Actualizar o insertar": si el mensaje ya existe, se actualiza; si no, se crea. Evita duplicados	La API Java al recibir mensajes
Programador de tareas	Una función del servidor que ejecuta un programa a una hora fija (en inglés, cron)	"Corre la ingesta cada hora"
Panel / dashboard	Una página web para revisar y aprobar borradores, y ver gráficos y alertas	El panel en Streamlit

4. El mapa general

Todas las piezas viven en un servidor desplegado. Solo el panel queda abierto a internet; la base de datos y la IA quedan privadas, y el bot no necesita recibir visitas porque es él quien se conecta a Discord.



Las tres reglas de la propuesta

- **Un solo formato de mensaje:** el contrato de la ingesta, que ya está programado y probado.
- **Una sola puerta y un solo registro:** todo entra por la API Java y se guarda en su base de datos. Eso permite tener el historial de cada persona, que el dashboard necesita.
- **La IA es un servicio que procesa:** recibe mensajes desde Java, devuelve resultados y no guarda la verdad del sistema.

5. Dónde se guarda cada dato

La base de datos y OCI no son lo mismo. **La base de datos guarda los datos con los que se trabaja todos los días** (se consultan y combinan); **OCI guarda los resultados finales** como archivos. El "historial de cada persona" no es un archivo aparte: son todas las filas de mensajes con el mismo autor en la base de datos.

Dato	Dónde	Quién lo escribe
Cada mensaje de Discord (autor, fecha, canal, texto...)	Base de datos	API Java, al recibirlo del bot o de la ingesta
Lo que la IA concluye de cada mensaje (intención, sentimiento, tema)	Base de datos, en la misma fila	API Java, al recibir la respuesta de la IA
Borradores y su aprobación (posts, FAQ)	Base de datos	API Java (curación)
Activos aprobados	OCI (requisito del brief)	API Java (ya lo hace)
Registros técnicos de la IA (logs)	OCI	IA (ya lo hace)
Respaldo de los lotes originales (opcional)	OCI	Ingesta o API Java

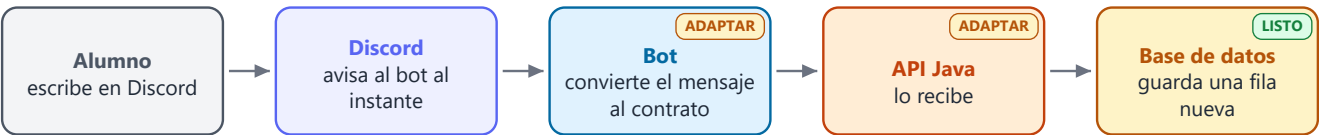
6. Cómo se cumple cada necesidad

Para cada necesidad: qué resuelve, el recorrido de la información, dónde queda guardada y qué tan lista está hoy.

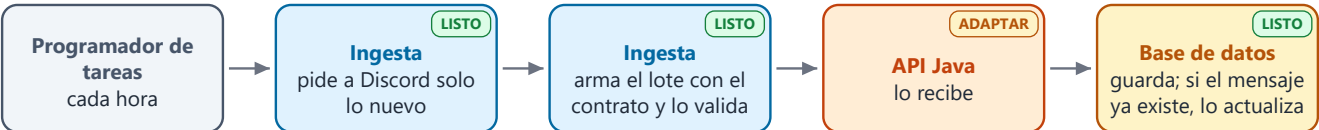
N1 · Capturar la actividad sin esfuerzo manual

Qué resuelve: que todo lo que pasa en Discord entre al sistema solo. Hay dos caminos: el vivo es rápido; el lote rescata lo que el bot se haya perdido (por ejemplo, si se reinició) y actualiza las reacciones que llegan después.

EN VIVO · EN SEGUNDOS



HISTORIAL · CADA HORA



Dónde queda guardado: en la base de datos, una fila por mensaje.

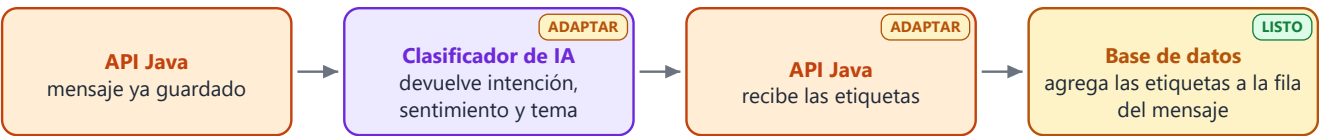
Qué tan listo está hoy

Pieza	Estado	Qué falta
Ingesta por lotes	LISTO	Nada: está programada y probada con los 39 mensajes de prueba
Bot	ADAPTAR	Existe (lo construyó IA). Falta que use el contrato, que no descarte a los alumnos simulados ni las capturas sin texto, y que envíe a la API Java
API Java	ADAPTAR	La puerta /process existe. Faltan los ajustes de la "opción C" (guía de la ingesta) y el upsert para no duplicar

N2 · Identificar al instante los logros y las dudas valiosas

Qué resuelve: que el sistema sepa qué es cada mensaje apenas llega.

CADA MENSAJE NUEVO



Según la intención, el mensaje sigue su camino:

Si la intención es...	Sigue en
TESTIMONIO (un logro)	N3 · borrador de post de LinkedIn y caso de éxito
PREGUNTA_FAQ (una duda)	N4 · respuesta del bot
COMENTARIO (una opinión)	N5 · alimenta el sentimiento del dashboard
OTRO (saludos, ruido)	Se descarta

Las dudas valiosas (por ejemplo, un debate con muchas respuestas) se detectan contando cuántos mensajes responden a una duda. El contrato ya trae ese dato (a qué mensaje responde cada uno), y el conteo lo hace el dashboard (N5).

Dónde queda guardado: en la base de datos, en la misma fila del mensaje.

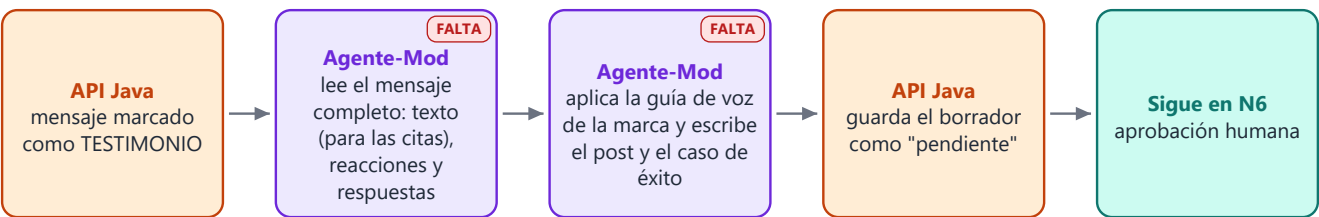
Qué tan listo está hoy

Pieza	Estado	Qué falta
Clasificador de IA	ADAPTAR	Hoy devuelve solo la intención. Falta que devuelva también el sentimiento y el tema
API Java	ADAPTAR	Ya tiene la columna de sentimiento. Falta la de tema y guardar las etiquetas que devuelve la IA

N3 · Posts de LinkedIn con la voz de la marca

Qué resuelve: convertir un logro en un borrador de post y de caso de éxito, listo para que Marketing lo revise.

CADA LOGRO



Dónde queda guardado: en la base de datos, como borrador pendiente de aprobación.

El post no se publica solo: el brief pide posts "listos para publicar", y la publicación la hace Marketing después de aprobarlos.

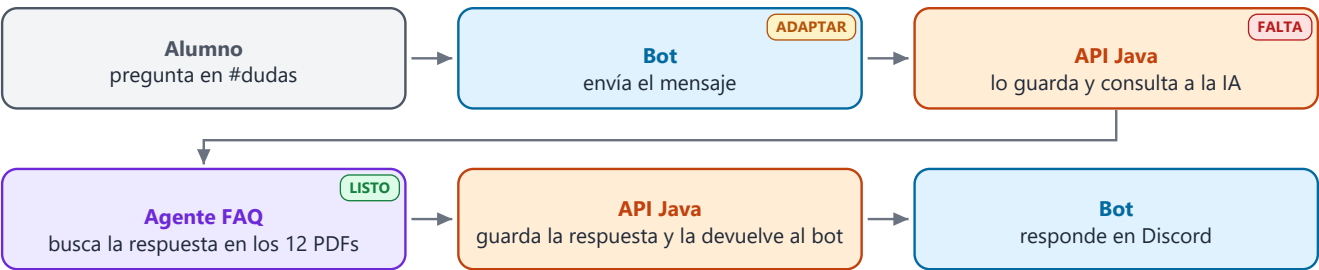
Qué tan listo está hoy

Pieza	Estado	Qué falta
Agente-Mod	FALTA	No existe en el repositorio. Es el trabajo más grande y el más importante para Marketing
Guía de voz de la marca	FALTA	Un texto con el tono de la institución. Para la demo puede escribirlo el equipo
Consentimiento del alumno	FALTA	Decidir cómo se pide antes de publicar el nombre o la cita de un alumno

N4 · FAQ y bot que responde dudas

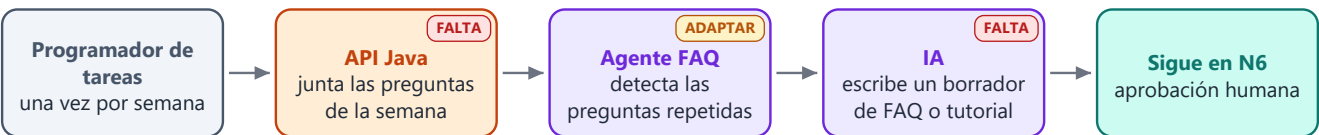
Qué resuelve: que el alumno reciba una respuesta al instante y que las dudas repetidas se conviertan en contenido.

RESPUESTA EN VIVO



Si el Agente FAQ...	Entonces
encuentra la respuesta en los PDFs	El bot responde con la respuesta y la cita del documento
no la encuentra	No inventa: el bot avisa "un mentor te responderá" y el panel muestra una alerta al CM

FAQ DINÁMICA · SEMANAL



Dónde queda guardado: en la base de datos: la pregunta, la respuesta y si requirió a una persona.

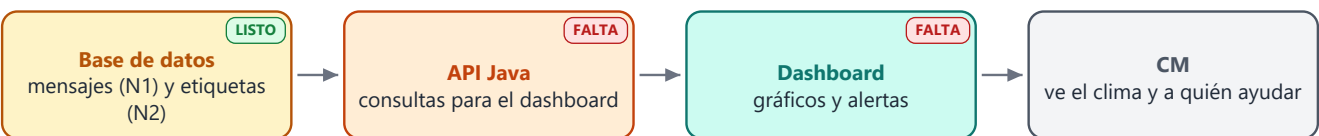
Qué tan listo está hoy

Pieza	Estado	Qué falta
Agente FAQ	LISTO	Nada: es la pieza más avanzada del proyecto
Bot	ADAPTAR	Que envíe a la API Java en lugar de ir directo a la IA
API Java	FALTA	Una puerta nueva "en vivo" que espera la respuesta de la IA. Hoy espera como máximo 15 segundos: hay que subirlo, porque la IA puede tardar más
FAQ dinámica	ADAPTAR	La IA ya detecta preguntas repetidas. Falta generar el borrador semanal

N5 · Dashboard de sentimiento, tendencias y alertas

Qué resuelve: que el CM vea el clima de la comunidad y sepa a quién ayudar. No necesita IA nueva: usa lo que N1 y N2 ya guardaron.

CADA VEZ QUE SE ABRE EL DASHBOARD



Indicador	Cómo se calcula	¿Usa IA?
Sentimiento por semana	Cuenta las etiquetas de sentimiento de los mensajes de cada semana	Sí, la etiqueta
Temas en tendencia	Cuenta cuántos mensajes hubo de cada tema por semana	Sí, la etiqueta
Alerta de deserción	Busca a quién no escribe hace más de 14 días (con el autor y la fecha de cada mensaje)	No
Alerta de frustración	Personas con varios mensajes negativos seguidos	Sí, la etiqueta

Indicador	Cómo se calcula	¿Usa IA?
Dudas sin responder	Preguntas que nadie respondió después de N horas	No

Dónde queda guardado: nada nuevo: el dashboard solo lee lo que ya está en la base de datos.

Qué tan listo está hoy

Pieza	Estado	Qué falta
Datos necesarios	LISTO	Nada: el contrato ya trae el autor estable, la fecha y a qué mensaje responde cada uno
Consultas en la API Java	FALTA	Unas 5 consultas, una por indicador
Dashboard en el panel	FALTA	Construirlo

N6 · Aprobación humana antes de publicar

Qué resuelve: que nada salga a internet sin que una persona lo revise.

CADA BORRADOR



Dónde queda guardado: en la base de datos: el borrador, quién lo editó y cuánto tardó la revisión (backend ya lo registra).

Qué tan listo está hoy

Pieza	Estado	Qué falta
API Java	LISTO	Nada: ya permite listar, editar y aprobar los paquetes
Panel de aprobación	FALTA	Construirlo

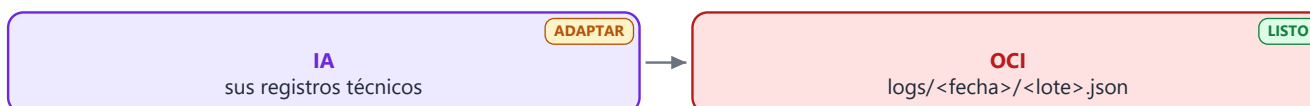
N7 · Guardar los activos en OCI

Qué resuelve: cumplir el requisito del brief de guardar los activos generados en un bucket de OCI.

CADA ACTIVO APROBADO



REGISTROS DE LA IA



Dónde queda guardado: en OCI, como archivos.

Qué tan listo está hoy

Pieza	Estado	Qué falta
Subida desde la API Java	LISTO	Nada: backend ya lo hace
Subida desde la IA	ADAPTAR	Hoy la IA sube también paquetes de activos. Que suba solo sus logs, para no tener dos versiones de lo mismo

7. Un solo mensaje que recorre las 7 necesidades

Camila escribe en #logros: "me contrataron!!!! después de 6 meses en el bootcamp y 40 postulaciones, empiezo el lunes como QA trainee. No se rindan"

Paso	Necesidad	Qué pasa
1	N1	El bot lo recibe al instante y la API Java lo guarda como una fila
2	N2	La IA lo etiqueta: TESTIMONIO, muy positivo, tema "empleo"
3	N3	El Agente-Mod escribe el borrador del post y del caso de éxito, citando a Camila
4	N1	Una hora después, el lote trae el mismo mensaje con 12 reacciones de festejo, y la API Java actualiza la fila (no la duplica)
5	N5	El dashboard suma un mensaje positivo y una contratación esta semana
6	N6	Marketing ve el borrador en el panel, corrige una frase y lo aprueba
7	N7	La API Java sube el post aprobado a OCI

8. Qué está listo y qué falta construir

Cada celda dice qué tan lista está una pieza para la necesidad de esa columna. Vacío: la pieza no participa.

Pieza	N1	N2	N3	N4	N5	N6	N7
Ingesta (lotes)	LISTO						
Bot	ADAPTAR			ADAPTAR			
API Java	ADAPTAR	ADAPTAR	ADAPTAR	FALTA	FALTA	LISTO	LISTO
Clasificador de IA		ADAPTAR					
Agente FAQ				LISTO			
Agente-Mod			FALTA				
Panel y dashboard					FALTA	FALTA	
OCI							LISTO

Lo más urgente

- **El Agente-Mod:** sin él no hay posts de LinkedIn ni casos de éxito, que son el corazón del negocio de Marketing.
- **El panel:** sin él no hay aprobación humana ni dashboard.
- **La puerta "en vivo" de la API Java:** sin ella el bot no puede pasar por Java.
- **Acordar los formatos entre equipos:** es lo más barato y destraba todo lo demás.

Despliegue

Pieza	¿Siempre encendida?	¿Abierta a internet?
Bot de Discord	Sí, las 24 horas	No: es el bot el que se conecta a Discord. Debe correr una sola copia, o cada mensaje se respondería dos veces
API Java y base de datos	Sí	La API, solo si el panel está en otro lugar. La base de datos, no
IA (orquestador)	Sí	No: solo la llama la API Java
Ingesta por lotes	No: corre cada hora y termina	No
Panel Streamlit	Sí	Sí: lo usan Marketing y el CM desde el navegador

Falta decidir dónde se despliega (el servidor del equipo o una máquina gratuita de OCI) y comprobar que tenga memoria suficiente: los modelos de búsqueda del Agente FAQ consumen bastante.

9. Preguntas para el equipo

Para cerrar la propuesta necesitamos estas respuestas:

#	Pregunta	Por qué importa
1	¿Validamos la propuesta 3 como arquitectura del proyecto?	Define qué cambia cada equipo
2	¿El bot pasa por la API Java (como en este documento) o va directo a la IA?	Pasar por Java garantiza que todo quede registrado y deja la IA privada; ir directo es un poco más rápido
3	¿Quién construye el Agente-Mod?	Sin él no hay posts de LinkedIn (N3)
4	¿Quién construye el panel y el dashboard?	Sin ellos no hay aprobación ni alertas (N5 y N6)
5	¿Alguien está trabajando en n8n? ¿Lo necesitamos?	En esta propuesta no hace falta: el bot y la API Java cumplen esa función
6	¿El bot de IA usa el mismo bot de Discord (token) que la ingesta?	Dos bots activos responderían dos veces
7	¿Dónde desplegamos y con qué recursos?	Todo debe estar desplegado; la IA necesita memoria suficiente
8	¿Cómo pedimos el consentimiento de los alumnos antes de publicar?	Publicar el nombre o la cita de un alumno usa sus datos personales

Documentos de referencia en el repositorio (rama feature/discord-ingestion): docs/INGESTION_GUIDE.md (comparación detallada con backend y opción C), docs/CONTRACT.md (el contrato campo por campo) y docs/PROJECT_BRIEF.md (el brief y el alcance del MVP).